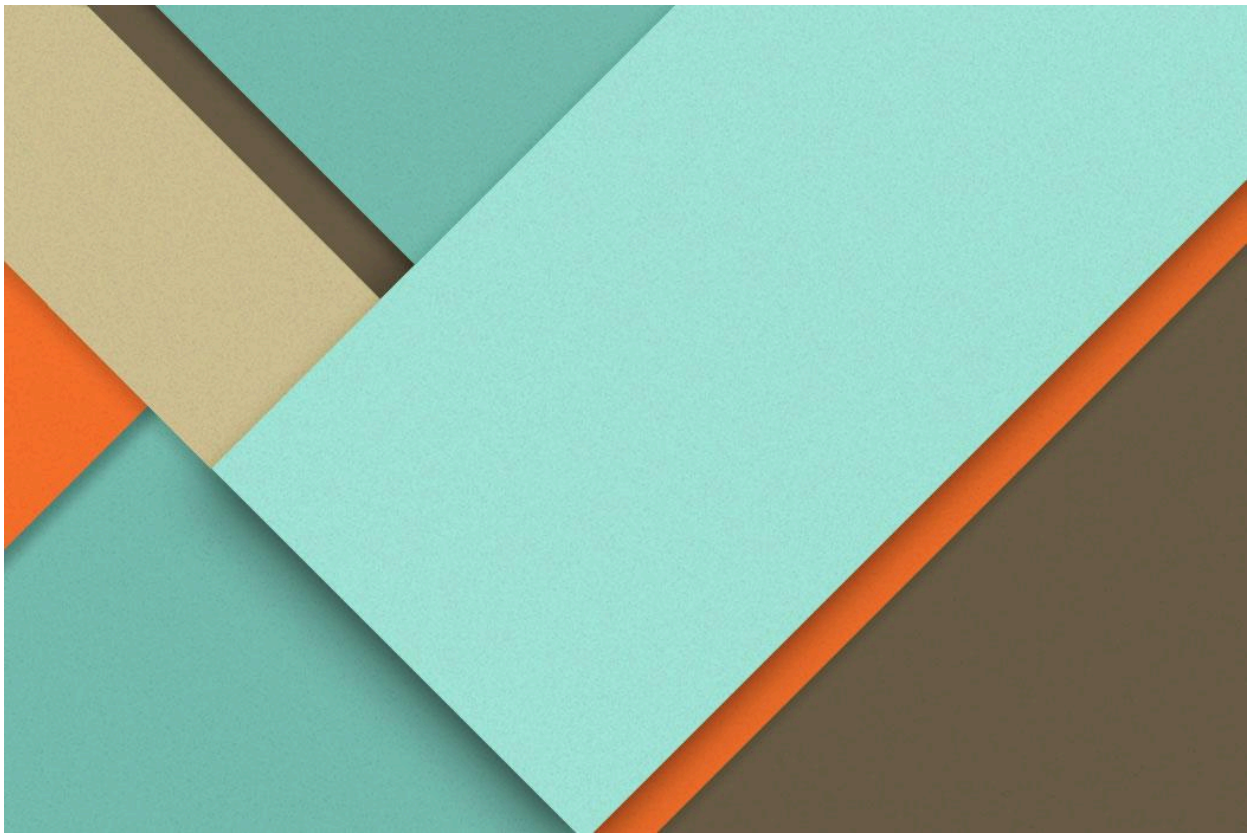


Secure Website Applications

Hafza Abdullahi: submission date - 30th march 2026

—

1. Reflective Cross-Site Scripting (XSS)....1
Vulnerability Knowledge.... 2
Mitigation Strategy....2
2. Session Fixation....2
Vulnerability Knowledge.... 3
Mitigation Strategy....3
3. OS Command Injection....3
Vulnerability Knowledge.... 4
Mitigation Strategy....4
4. Directory Traversal.... 4
Vulnerability Knowledge.... 5
Mitigation Strategy....**5**
5. Insufficient Session Management (Direct Page Access)....5
Vulnerability Knowledge.... 6
Mitigation Strategy....6
5b. Insufficient Session Management.... 6
Vulnerability Knowledge.... 7
Mitigation Strategy....7
6. Persistent Cross-Site Scripting (XSS)....7
Vulnerability Knowledge.... 8
Mitigation Strategy....8
7. Brute-Force Protection (Anti-Automation).... 9
Vulnerability Knowledge.... 10
Mitigation Strategy....10
8. SQL Injection (Displaying DB Error to front end)....10
Vulnerability Knowledge.... 11
Mitigation Strategy....11
9. Cross-Site Request Forgery (CSRF)....11
Vulnerability Knowledge.... 12
Mitigation Strategy....12
10. Page Caching (Information Leak via Back Button).... 12
Vulnerability Knowledge.... 13
Mitigation Strategy....13

1. Reflective Cross-Site Scripting (XSS)

Location: http://localhost/project26/includes/login.inc.php.

Target: http://localhost/project26/includes/index.php.

Parameter: uid

Payload:
`uid="><script>alert('Reflected_XSS')</script>&pwd=password&submit=login`

Vulnerability Knowledge

- **Classification:** Reflected XSS
- **Root Cause:** The application handled failed logins by taking the raw, unvalidated username (uid) from the POST request and storing it in a session variable (`$_SESSION['failedMsg']`). This variable was then directly echoed onto the homepage (index.php).
- **Explanation:** Since the input is reflected back to the user without any output encoding, any malicious script will also be reflected. The browser reads this unsanitized reflection as active HTML/JavaScript, allowing the harmful code to execute in the context of the user's session.

Mitigation Strategy

- **Control used:** Contextual output encoding via htmlspecialchars()
- **Implementation:** I updated the custom cleanChars() function in login.inc.php to utilize PHP's built-in htmlspecialchars($val, ENT_QUOTES, 'UTF-8'). This wrapper function is now applied to the $uid before it is concatenated into the $_SESSION['failedMsg'] string.

Result: The browser now successfully converts special characters into HTML entities (< becomes <). Inspecting the page source confirmed the tags were neutralized, and the webpage rendered the payload as literal text, preventing execution.

Trade offs: The main trade off for output encoding is the inability to render intended html. If the page needed to display rich text formatting (bold, italics) submitted by a user, htmlspecialchars would break that functionality. However for a login form this is not an issue.

2. Session Fixation

Location: http://localhost/project26/includes/login.inc.php

Target: User Session Management (`PHPSESSID` cookie)

Problem: The application authenticates the user but does not update the `PHPSESSID` cookie. The unauthenticated session ID is simply upgraded to an authenticated session ID.

Vulnerability Knowledge

- **Classification:** Session Fixation
- **Root Cause:** The application fails to issue a new, secure session identifier once a change in privilege level (logging in) occurs.
- **Explanation:** Somoene can steal and use a known session ID (e.g., by sending them a crafted link). When the victim logs in, the session becomes authenticated, but the ID remains the same. The attacker, already knowing this ID, can now use it to hijack the victim's newly authenticated session without ever needing their username or password.

Mitigation Strategy

- **Control used:** Session ID Regeneration
- **Implementation:** I added `session_regenerate_id(true);` function to the `login.inc.php` script, placing it immediately after the user's credentials are verified but before the authenticated `$_SESSION` variables are set.

Result: When the user successfully logs in, the server generates a brand new `PHPSESSID` cookie and completely destroys the old session file (enforced by the `true` boolean). Even if the attacker knew the old session ID, it is now useless.

Trade-offs: There is a small overhead anytime a new session ID is generated. On complex websites with a lot of background tasks can cause race conditions when generating session IDs. Glitches can happen if a background task tries to use the old ID as it's being destroyed.

3. OS Command Injection

Location: http://localhost/project26/includes/auth1.php

Target: Underlying Server Operating System

Payload: localhost && dir

Problem: By adding system commands to the expected IP address (`localhost && dir`), the server executes the commands and returns the output to the browser, instead of treating it a input string

Vulnerability Knowledge

- **Classification:** Injection / OS Command Injection
- **Root Cause:** The application uses the `shell_exec()` function to run a system `ping` command, but it chains the raw, unvalidated user input (`$_REQUEST['target']`) directly into the command string.
- **Explanation:** Because the input is not sanitized, anyone can use shell metacharacters (like `&&`, `;`, or `|`) to terminate the intended `ping` command and add their own malicious commands. The operating system executes these injected commands with the privileges of the web server.

Mitigation Strategy

- **Control used:** Input Sanitization via Shell Escaping.
- **Implementation:** I wrapped the user's input variable in PHP's built-in `escapeshellarg()` function before passing it to `shell_exec()`.

Result: `escapeshellarg()` adds single quotes around a string and quotes/escapes any existing single quotes within it. This forces the operating system to interpret the entire user payload as a single literal argument to the `ping` command, making all shell metacharacters completely harmless.

Trade-offs: The primary trade-off is a minor loss of edge-case functionality. If an advanced user needed to pass specific `ping` flags (`-t` or `-n`) alongside the IP address in that single input box, `escapeshellarg()` would break that functionality by treating the flags as part of the hostname rather than executable arguments.

4. Directory Traversal

Location: http://localhost/project26/auth2.php

Target: ../../../../windows/system32/calc.exe

Parameter: FileToView

Problem: : By using dot and slash (`../`) characters in the `FileToView` parameter in the URL, an attacker can escape the web directory and read sensitive files from the operating system.

Vulnerability Knowledge

- **Classification:** Broken Access Control / Directory Traversal
- **Root Cause:** The application extracted unvalidated user input directly from the URL (`$_GET['FileToView']`) and passed it directly into the `file_get_contents()` function without restricting the file path.
- **Explanation:** Without input sanitization, the server blindly trusts the path provided by the user. The `../` string is a standard filesystem command, used to move up one directory. By chaining these together (`../../../../`), an attacker navigates to the root drive and accesses highly restricted configuration or password files.

Mitigation Strategy

- **Control used:** Input Sanitization (Path Stripping)
- **Implementation:** I wrapped the user's input in the `basename()` function before interacting with the file system. I also replaced `file_get_contents()` check with a safer `file_exists()` check, and wrapped the final output in `htmlspecialchars()` as a defense-in-depth measure against Stored XSS inside the text files.

Result: The `basename()` function inherently strips out all directory path information (including all slashes and `..` characters), leaving only the actual file name. For example, `../../System32/calc.exe` is instantly reduced to `calc.exe` The application then checks if that exact file exists only within the current, authorized web directory, removing the traversal attempt

Trade-offs: The main trade-off is a restriction on folder structure, because `basename()` removes all folder paths, all legitimate text files the application intends to read *must* be stored in the exact same directory as `auth2.php`. Files can no longer be organized by text files into subfolders (e.g., a URL like `?FileToView=documents/yellow.txt` will break, as `basename()` will strip `documents/` and fail to find the file.

5. Insufficient Session Management (Direct Page Access)

Location: http://localhost/project26/admin.php

Target: Admin Control Panel

Problem: Without authentication, the admin page is accessible to anyone by directly typing the URL into the browser.

Vulnerability Knowledge

- **Classification:** Broken Access Control / Missing Function Level Access Control.
- **Root Cause:** The `admin.php` file contained an empty conditional check for session variables. While it verified if a user was an admin (`$_SESSION['u_admin'] == 0`), it lacked the necessary commands inside the `if` statement to actually stop the page from loading.
- **Explanation:** Access control was not enforced on the server side. An unauthenticated user could bypass the login entirely by typing the direct URL of the admin panel into their browser, gaining unauthorized access to the application's sensitive login logs.

Mitigation Strategy

- **Control used:** Server-Side Access Control Enforcement
- **Implementation:** I updated the validation block at the top of `admin.php`, If a user session does not exist, or if the `u_admin` session flag evaluates to false, the application executes a `header("Location: index.php")` redirect, immediately followed by the `exit()` function.

Result: The `exit()` function strictly stops the script, kicking any unauthorized user and redirecting them back to the homepage before the HTML can even load.

Trade-offs: The main downside is that this fix must be manually inserted at the top of every single private page built. If it's forgotten even once, that specific page is left completely unprotected.

5b. Insufficient Session Management

Location: http://localhost/project26/includes/logout.inc.php

Target: User Session Management

Problem: The logout function did not completely destroy the user's session footprint on the client side.

Vulnerability Knowledge

- **Classification:** Broken Authentication / Session Management.
- **Root Cause:** The original logout script only cleared variables and destroyed the session file on the server (`session_destroy()`), but left the active `PHPSESSID` cookie alive inside the user's browser**.**
- **Explanation:** If the session cookie is not explicitly destroyed, the browser retains the token. This can lead to zombie sessions where the browser attempts to access the session state, especially when combined with caching vulnerabilities or "Back" button navigation.

Mitigation Strategy

- **Control used:** Complete Session Destruction
- **Implementation:** I refactored the logout script to perform a destruction process:
    - unsetting the `$_SESSION` array,
    - explicitly overwriting and expiring the `PHPSESSID` cookie in the user's browser using `setcookie()`,
    - destroying the server session, and redirecting the user.

Result: The session footprint is completely destroyed from both the server and the client's browser, ensuring complete loss of access to previous pages.

Trade-offs: There are no main negative trade-offs to properly destroying a cookie, except a slight increase in code in the logout script.

6. Persistent Cross-Site Scripting (XSS)

Location: http://localhost/project26/includes/login.inc.php.

Target: http://localhost/project26/admin.php

Parameter: uid

Vulnerability Knowledge

- **Classification:** Injection / Cross-Site Scripting (Stored/Persistent).
- **Root Cause:** The application failed to sanitize or encode the user data before storing it in the loginEvents database table, and also failed to encode the data when displaying it on the admin panel.
- **Explanation:** When a malicious script like `<script>alert(1)</script>` in the uid field, the application saves this raw HTML/JavaScript to the database. When an admin later looks at **admin.php**, the browser fetches this string and executes it as code rather than displaying it as text.

Mitigation Strategy

- **Control used:** Defence in Depth (Input Sanitization + Output Endcoding)
- **Implementation:** First, I applied the `cleanChars()`sanitization function to the data before the INSERT SQL statement in login.inc.php. Second, to ensure malicious data couldn't execute, I implemented Output Encoding on **admin.php**. I wrapped all database variables **(IP, Timestamp, User ID, Outcome)** in `htmlspecialchars(..., ENT_QUOTES, 'UTF-8')` immediately before they are echoed into the HTML form values.

Result: The browser now successfully converts special characters into HTML entities **(< becomes <)**. Looking at the response from zap and the what the html page loaded confirmed the tags were neutralized, and the webpage rendered the payload as literal text, preventing execution.

Trade offs

- **Data Integrity & Debugging:** The raw payload is permanently altered in the database. If a database administrator looks at the logs, they will only see the escaped HTML entities (e.g., `<script>`) rather than the true data, complicating forensic analysis.
- **Multi-Media Output Issues:** If the application later needs to export these logs to a different format that isn't HTML (such as a PDF report, a CSV file, or a JSON API), the escaped payload in the db will have to actively *un-escape* the data first, which adds complexity.
- **Double Encoding:** By applying filtering on both input and output, it can cause issues. If a framework automatically escapes the output, but the data was already manually escaped on input, it results in "double encoding" (e.g., displaying `<script>`), which degrades the user interface.

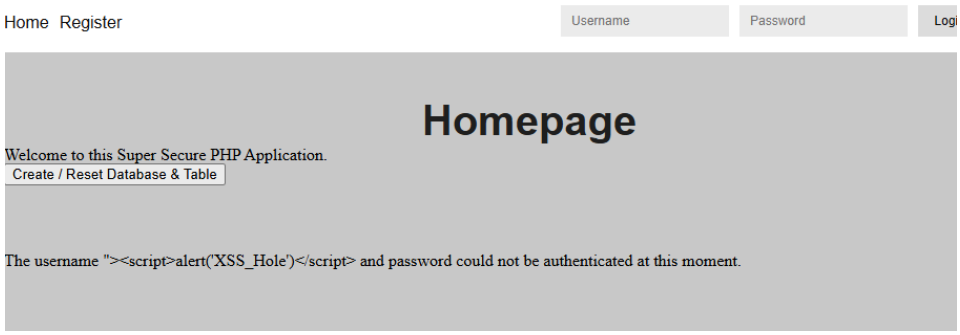
Properly Escaped HTML code viewed from inside ZAP response:

```
"Create / Reset Database & Table">
                                <br><br><br>

    </form>
                    <br><br>The username &quot;&gt;&lt;script&gt;alert(&#039;
XSS_Hole&#039;)&lt;/script&gt; and password could not be authenticated at this
moment.
            </div>
        </section>


    </body>
```

Escaped HTML code viewed from front end:



7. Brute-Force Protection (Anti-Automation)

Location: http://localhost/project26/includes/signup.inc.php

Target: Account Registration

Problem: The registration page allowed an unlimited number of registration attempts. While the login page has a lockout mechanism, the signup page is completely unprotected, allowing a bot to flood the database with fake accounts.

Vulnerability Knowledge

- **Classification:** Broken Anti-Automation / Insufficient Rate Limiting.
- **Root Cause:** Inconsistent security logic. Rate limiting was applied to the login logic but neglected on the registration logic.
- **Explanation:** Without a throttle, anyone can use a script to send thousands of "Create Account" requests per minute. This can flood the database with junk data or allow an attacker to "reserve" thousands of usernames, preventing real users from signing up.

Mitigation Strategy

- **Control used:** IP-Based Rate Limiting with Dynamic Lockout
- **Implementation:** I updated the security logic by using the same logic from the login script into the signup script. The application now queries the `failedLogins` table before processing any new registration. If an IP address has recorded 5 or more attempts, the script calculates the time difference.

Result: If the attempts occurred within the last 180 seconds, the script stops the process and returns an error message to the user. This kills automated scripts by forcing them into a 3 minute time-out after every 5 requests.

Trade-offs: The main trade-off is the risk of "False Positives." If hundreds of students are sharing one public IP address at a university, one person spamming the registration button could accidentally lock out every other student on that campus from creating an account for 3 minutes.

8. SQL Injection (Displaying DB Error to front end)

Location: http://localhost/project26/includes/login.inc.php.

Target: Backend Database (`sapusers` table)

Parameter: uid

Payload:
`uid="><script>alert('XSS_Hole')</script>&pwd=password&submit=login`

Vulnerability Knowledge

- **Classification:** Injection / Error - Based SQL Injection
- **Root Cause:** In the original codebase, the `uid` and `pwd` parameters were concatenated directly into the SQL string within the `processLogin()` function `SELECT * FROM sapusers WHERE user_uid = '" .$uid. "'...`
- **Explanation:** During XSS testing, injecting a payload that contained double quotes (`"`) inadvertently broke the structure of the backend SQL query. The server responded with a **MariaDB error,** revealing sensitive database information to the frontend. This error proves that the database is executing user input as part of the query logic rather than treating it as literal text. A malicious actor could exploit this using payloads (like `' OR 1=1 --`) to completely bypass authentication

Mitigation Strategy

- **Control used:** Parameterized Queries (Prepared Statements).
- **Implementation:** I refactored the database queries in **`login.inc.php`** to completely remove direct string concatenation. I used prepared SQL statements (`$stmt = $conn->prepare("SELECT * FROM sapusers WHERE user_uid = ?")`) and then bound the user input to the query securely using `$stmt->bind_param("s", $uid)`.

Result: The database now pre-compiles the SQL logic *before* inserting the user's data. Even if a user types a malicious SQL command or special characters like quotes, the database strictly treats it as literal text and will fail to find a matching username, neutralizing the injection threat.

Trade Offs: Prepared statements introduce a slight increase in code verbosity, requiring the need for multiple lines of code (`prepare`, `bind`, `execute`) rather than a single concatenated string. Additionally, depending on the database, prepared statements can sometimes require two network round-trips to the database server (one to send the query template, and one to send the parameters), which introduces a very minor overhead compared to a standard query**.**

9. Cross-Site Request Forgery (CSRF)

Location: http://localhost/project26/includes/reset.inc.php

Target: Password Reset

Problem: The application generates a unique token (`csrftoken`) and includes it in the reset form, but the server-side script fails to actually validate this token. It processes any request as long as the password rules are valid.

Vulnerability Knowledge

- **Classification:** Broken Anti-Automation / Insufficient Rate Limiting.
- **Root Cause:** Failure to verify the authenticity of a state changing request. The server assumes that any request arriving with valid session cookies is trustworthy, without checking if the request originated from the application's own trusted forms.
- **Explanation**: CSRF is an attack that tricks a victim's browser into performing an unwanted action on a different website where they are currently logged in. Since the browser automatically sends the user's session cookies, the server cannot tell the difference between a legit "Reset Password" click from a malicious one. By not validating the token, the server allows attackers to forge password reset requests pretending to be a legit user.

Mitigation Strategy

- **Control used:** Synchronizer Token Pattern
- **Implementation:** I implemented a strict comparison check at the start of the `reset.inc.php` logic. The script now gets the `csrftoken` from the `$_GET` parameters and compares it directly against the secret `csrftoken` stored in the user's `$_SESSION`.

Result: If the tokens don't match, or if the token is missing, the script executes an `exit()` command and redirects the user to the homepage with a "Security Error." This ensures that only requests originating from the user's actual session modify the database entries.

Trade-offs: the possibility for stale tokens. If a user leaves the password reset page open for a long time and their session expires, or if they use the "Back" button across multiple tabs, the token in the URL may no longer match the session. This results in a security error for a legitimate user, making them refresh and restart the process.

10.Page Caching (Information Leak via Back Button)

Location: http://localhost/project26/includes/header.php

Target: Local Browser Cache & Back-Forward Cache (bfcache)

Problem: Authenticated pages are cached in the browser's memory, allowing a user to view sensitive data via the "Back" button after logging out.

Vulnerability Knowledge

- **Classification:** Security Misconfiguration / Insufficient Cache Controls.
- **Root Cause:** The server did not send strict HTTP response headers specifying how the client's browser should handle the storage of sensitive page data. Additionally, modern browsers use "bfcache" to freeze the page state, bypassing server checks.
- **Explanation:** By default, browsers cache HTML pages to speed up load times. If a user logs out on a shared or public computer, the next person to sit down can click the browser's "Back" button. This will load the sensitive page directly from its local cache without checking with the server to see if the user is still authenticated.

Mitigation Strategy

- **Control used:** HTTP Cache-Control Headers & Client-Side Cache Busting.
- **Implementation:** I added strict cache-busting headers in **header.php**
    - `Cache-Control: no-cache, no-store, must-revalidate`,
    - `Pragma: no-cache`
    - `Expires: 0`)

To combat modern browser bfcache, I also implemented a JavaScript `window.onpageshow` event listener that detects if the page was loaded from cache (`event.persisted`) and forces a hard reload.

Result: The browser is strictly forbidden from relying on a saved copy of the page. Clicking "Back" forces a fresh request to the server, which rejects the unauthenticated user and redirects them to the login screen.

Trade Offs: The main trade-off is a slight increase in bandwidth usage and slower page load times for the user. Because the browser cannot cache the HTML structure, it must re-download the page from the server on every visit.

logout.inc.php

As shown in the attached screenshot from the Microsoft Edge Application/Cache panel, the security headers are functioning as intended after logging out and attempting to access previous sensitive pages.

Back/forward cache

Not served from back/forward cache: to trigger back/forward cache, use Microsoft Edge's back/forward buttons, or use the test button below to automatically navigate away and back.

URL http://localhost/project26/index.php

Frames 2 issues found in 1 frame.

(2) http://localhost/project26/index.php

Test back/forward cache

Actionable

Pages with cache-control:no-store header cannot enter back/forward cache.

CacheControlNoStoreCookieModified

1 frame

Not Actionable

Internal error.

BrowsingInstanceNotSwapped

1 frame

Learn more: back/forward cache eligibility